

Esport-Manager Une plateforme pour la gestion de compétitions de jeux vidéo

1. Contexte du projet

Le monde de l'esport amateur est en pleine effervescence, mais son organisation repose encore trop souvent sur des outils dispersés et manuels (tableurs, Discord, etc.). Cette dispersion est une source fréquente d'erreurs, de retards et de frustration pour les organisateurs comme pour les joueurs. Mon projet, "Esport-Manager", a pour ambition de résoudre ce problème en proposant une solution logicielle centralisée et robuste.

L'objectif est de développer le **cœur applicatif (back-office)** de cette solution. La première itération prendra la forme d'une application de bureau développée en **Java (avec JavaFX ou Swing)**, permettant à plusieurs utilisateurs de collaborer via une base de données commune, posant ainsi les bases d'une future application web.

2. Les Acteurs du Système

Le fonctionnement de la plateforme s'articulera autour de trois rôles principaux, chacun disposant de droits et de fonctionnalités spécifiques :

- **L'Administrateur** : Le garant de l'intégrité de la plateforme.
 - Il supervise l'ensemble des utilisateurs (organisateur, joueurs) et peut gérer leurs comptes.
 - Il enrichit la plateforme en gérant le catalogue des jeux vidéo éligibles pour les tournois.
 - Il intervient en dernier recours pour la modération des contenus et la gestion des abus.
- **L'Organisateur** : Le chef d'orchestre de l'événement.
 - Il conçoit et paramètre son tournoi de A à Z : choix du jeu, définition des règles, des dates, et du format.
 - Il est le maître des inscriptions, validant les joueurs et les équipes participants.
 - Pendant l'événement, il supervise le bon déroulement, valide les scores des matchs et peut envoyer des annonces à tous les participants.
- **Le Joueur** : Le cœur de la compétition.
 - Il personnalise son profil, crée ou rejoint une équipe pour participer aux événements.
 - Il explore les tournois disponibles et s'y inscrit.
 - Il est un acteur clé du tournoi : il consulte ses matchs, rapporte ses résultats, et suit sa progression dans l'arbre de compétition.

3. Au Cœur des Fonctionnalités

Pour donner vie à ce système, plusieurs modules fonctionnels seront développés. Les fonctionnalités suivantes constituent le socle de l'application :

- **Authentification et Profils Utilisateurs** : Chaque acteur du système sera identifié par un compte unique (identifiant/mot de passe). Son profil regroupera ses informations personnelles (pseudo, email) et ses statistiques clés (tournois joués, ratio de victoires).
- **Gestion des Équipes** : Un joueur pourra créer sa propre équipe et inviter d'autres joueurs à le rejoindre via un système de notifications. La gestion d'une équipe est collaborative, permettant au capitaine de l'inscrire à des tournois adaptés.
- **Le Cycle de Vie d'un Tournoi** : La création d'un tournoi est un processus complet. L'organisateur le définit (jeu, places, etc.), ouvre les inscriptions, et une fois celles-ci closes, le système **génère automatiquement l'arbre de compétition (le bracket)**. Le statut du tournoi évoluera (Ouvert, En cours, Terminé) pour refléter sa progression.
- **Suivi des Matches et des Résultats** : La validation d'un match est interactive. Un joueur soumet le score, et l'organisateur le valide. Cette validation déclenche la mise à jour automatique du bracket, faisant avancer le joueur ou l'équipe victorieuse au tour suivant.
- **Un Système de Notifications Dynamique** : Pour assurer la fluidité des interactions, un centre de notifications est essentiel. Il alertera les utilisateurs des événements importants. Pour illustrer son rôle, voici quelques exemples :
 - Un organisateur est notifié de chaque **nouvelle inscription**.
 - Un joueur reçoit une confirmation lorsque son **inscription est validée**.
 - Un capitaine d'équipe est informé quand un joueur **accepte son invitation**.
 - Les participants d'un match sont prévenus que **leur prochain adversaire est connu**.

4. Vision Architecturale

Le projet sera développé avec une vision à long terme. L'architecture logicielle sera conçue pour être **fortement modulaire**. Cette approche a un double avantage :

1. Elle facilitera la maintenance et les tests de chaque partie de l'application.
2. Elle garantira une évolution aisée vers une application web. Le "cœur" de l'application (la couche logique) pourra être réutilisé avec un minimum de modifications, en remplaçant simplement l'interface JavaFX par une interface web.